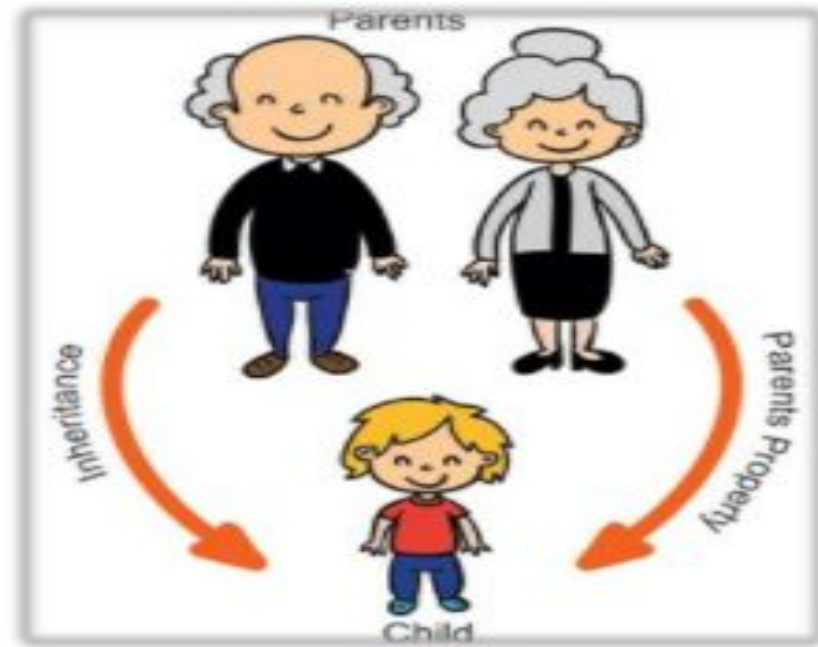


unit III

inheritance, packages and interfaces

inheritance



inheritance

- Inheritance is an important feature of OOP
- The idea behind inheritance in Java is that we can create new classes that are built upon existing classes.
- When we inherit from an existing class, we can reuse methods and data of the parent class.
- Moreover, we can add new methods and data in the new class also.
- Inheritance represents the IS-A relationship which is also known as a parent-child relationship.

inheritance

Terms used in Inheritance

Class: A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.

Sub Class/Child Class: Subclass is a class which inherits the other class. It is also called a derived class, sub class, or child class.

Super Class/Parent Class: Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.

Reusability: As the name specifies, reusability is a mechanism which facilitates us to reuse the data and methods of the existing class when we create a new class.

inheritance

The syntax of Java Inheritance:

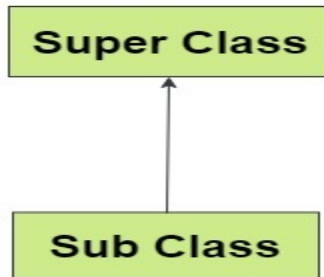
```
class Subclass-name extends Superclass-name  
{  
    //methods and fields  
}
```

The **extends keyword** indicates that we are creating a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.

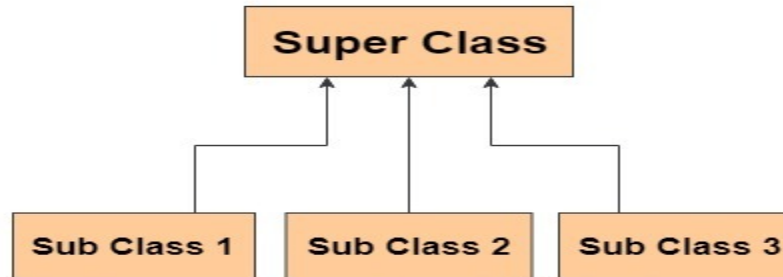
In the terminology of Java, a class which is inherited is called a parent or superclass, and the new class is called child or subclass.

Types of inheritance

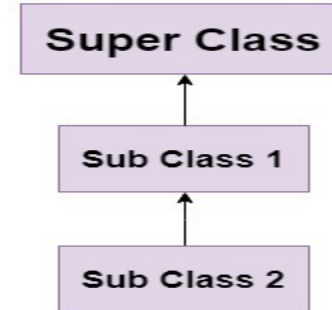
Single Inheritance



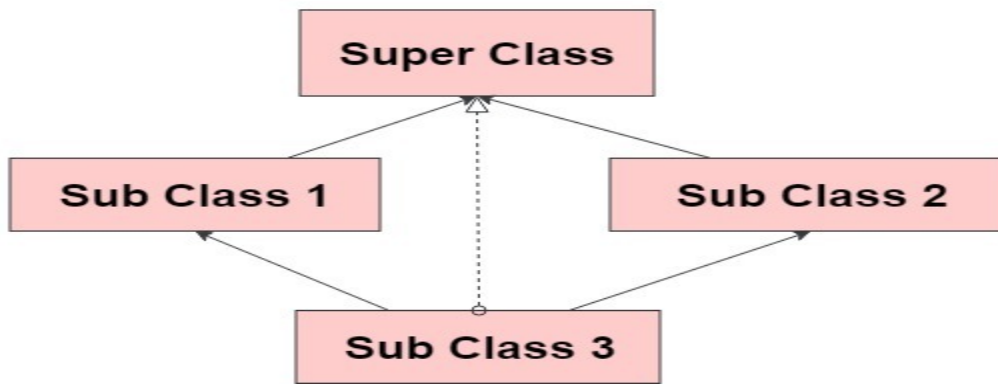
Hierarchial Inheritance



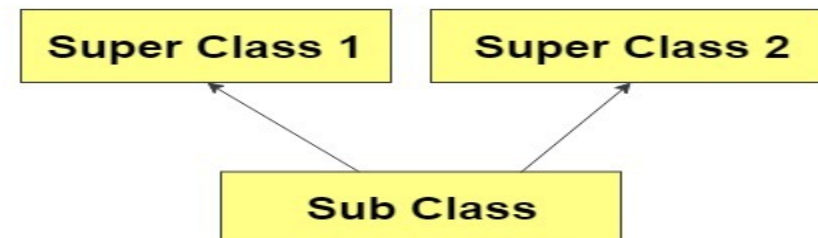
MultiLevel Inheritance



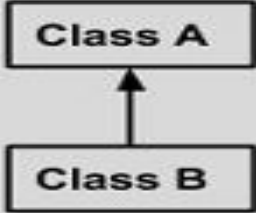
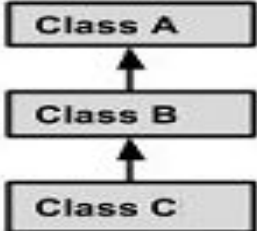
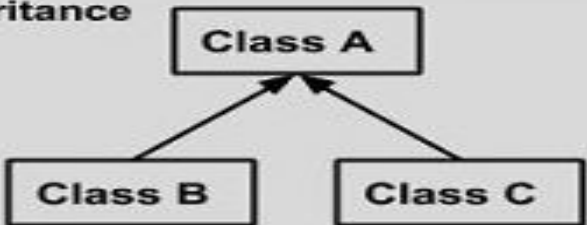
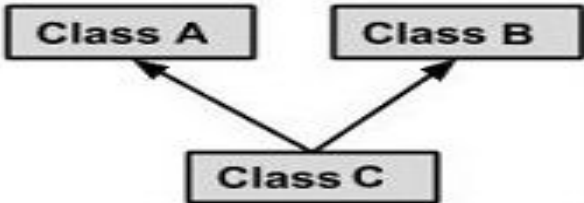
Hybrid Inheritance



Multiple Inheritance



Types of inheritance

Single Inheritance	 <pre>graph BT; B[Class B] --> A[Class A]</pre>	<pre>public class A { } public class B extends A { }</pre>
Multi Level Inheritance	 <pre>graph BT; C[Class C] --> B[Class B]; B --> A[Class A]</pre>	<pre>public class A {} public class B extends A {.....} public class C extends B {.....}</pre>
Hierarchical Inheritance	 <pre>graph BT; B[Class B] --> A[Class A]; C[Class C] --> A</pre>	<pre>public class A {} public class B extends A {.....} public class C extends A {.....}</pre>
Multiple Inheritance	 <pre>graph BT; C[Class C] --> A[Class A]; C --> B[Class B]</pre>	<pre>public class A {} public class B {.....} public class C extends A,B { } // Java does not support mutiple Inheritance</pre>

agenda

- Method Overriding
- Super and Final Keyword
- Example

When Constructors Are Executed???

- When a class hierarchy is created, in what order are the constructors for the classes that make up the hierarchy executed?
- For example, given a subclass called B and a superclass called A, is A's constructor executed before B's, or vice versa?
- **The answer is that in a class hierarchy, constructors complete their execution in order of derivation, from superclass to subclass.**
- Further, since `super( )` must be the first statement executed in a subclass's constructor, this order is the same whether or not `super( )` is used.
- If `super( )` is not used, then the default or parameterless constructor of each superclass will be executed.

Super keyword

Usage of Super Keyword

1

Super can be used to refer immediate parent class instance variable.

2

Super can be used to invoke immediate parent class method.

3

super() can be used to invoke immediate parent class constructor.

Method overriding

- If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java**.
- A method in the subclass has the same name and type signature as a method in its super class, then the method in the subclass is said to override the method in the superclass
- In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.
- Usage of Java Method Overriding
 - ✓ Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.
 - ✓ Method overriding is used for runtime polymorphism

Method overriding

□ Rules for Java Method Overriding

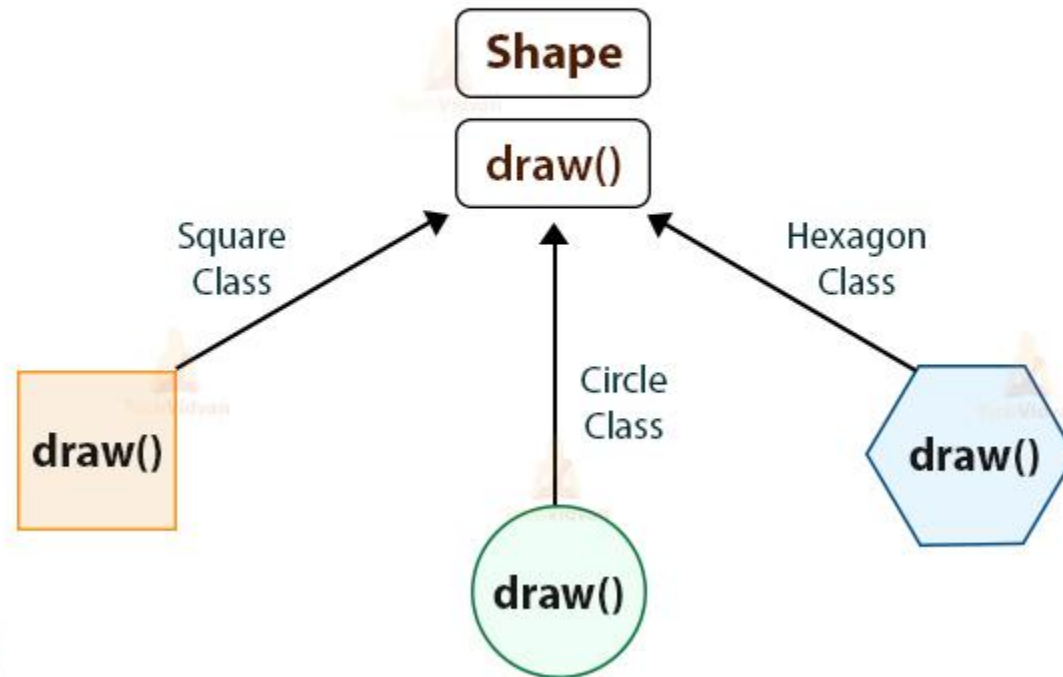
- ✓ The method must have the same name as in the parent class
- ✓ The method must have the same parameter as in the parent class.
- ✓ There must be an IS-A relationship (inheritance).

□ When an overridden method is called from within a subclass, it will always refer to the version of that method defined by the sub class

□ The method in superclass will be hidden

Method overriding

Method Overriding in Java



Method overriding

```
class Cat{  
    public void Sound(){  
        System.out.println("meow");  
    }  
}  
class Lion extends Cat{  
    public void sniff(){  
        System.out.println("sniff");  
    }  
    public void Sound(){  
        System.out.println("roar");  
    }  
}
```

Overriding

**Same method
name and same
parameters**

final keyword

Java **Final** Keyword

Final Variable



Stop value change

Final Method



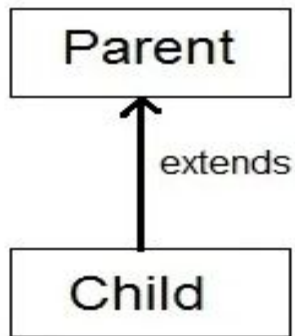
Prevent Method Overriding

Final Class



Prevent Inheritance

Dynamic method dispatch



```
Parent p = new Parent( );
```

```
Child c = new Child( );
```

```
Parent p = new Child( );
```

Upcasting

```
Child c = new Parent( );
```

incompatible type

When **Parent** class reference variable refers to **Child** class object, it is known as **Upcasting**.

Dynamic method dispatch

- Method overriding forms the basis for one of Java's most powerful concepts: dynamic method dispatch.
- Dynamic method dispatch is a mechanism by which a call to an overridden method is resolved at runtime rather than compile time.
- This is how java implements runtime polymorphism.
- When an overridden method is called by a reference, java determines which version of the method to execute based on the type of object it refers to.
- In simple words the type of object which it refers to determines which version of overridden method will be called.

Dynamic method dispatch

```
class A
{
    void callme()
    {
        System.out.println("Inside A");
    }
}
class B extends A
{
    void callme()
    {
        System.out.println("Inside B");
    }
}
class C extends A
{
    void callme()
    {
        System.out.println("Inside C");
    }
}

class Dispatch
{
    public static void main(String args[])
    {
        A a = new A();
        B b = new B();
        C c = new C();
        A r; //Superclass reference
        r = a; //r refers to an A object
        r.callme(); //calls A's version of callme
        r = b;
        r.callme();
        r = c;
        r.callme();
    }
}
```

Abstract classes and methods

- The *abstract* keyword is used to declare a class or a method as **abstract**.
- Abstract classes cannot be instantiated that is one cannot create an object of abstract class. Whereas, a reference can be created for an abstract class, but they can be subclassed.
- An abstract method is a method that is declared without an implementation.
- An **abstract class** is a class that is declared abstract means it may or may not include abstract methods.

Abstract methods-why ??

- Sometimes, the method definition will not be having any meaning in superclass.
- Only the subclass (specialization) may give proper meaning for such methods.
- In such a situation, having a definition for a method in superclass is absurd.
- Also, we should enforce the subclass to override such a method.
- A method which does not contain any definition in the superclass is termed as abstract method.
- Such a method declaration should be preceded by the keyword abstract.

Abstract classes and methods

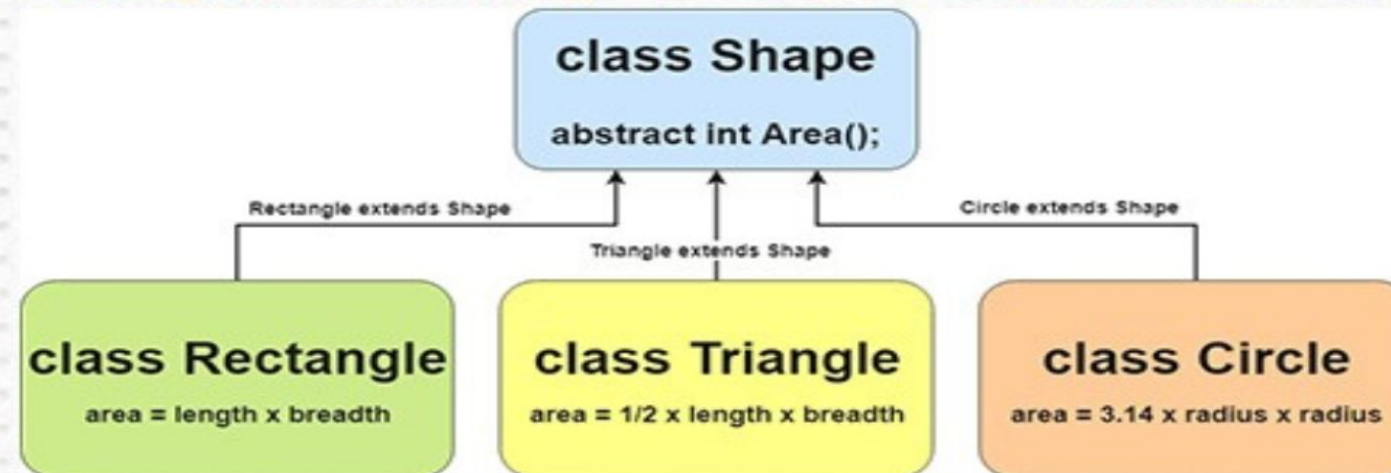
```
abstract class Animals
{
    private String name;
    // All kind of animals eat food to make this
    common to all animals
    public void eat()
    {
        System.out.println(" Eating .....");
    }
    // The animals make different sounds. They
    will provide their own implementation
    abstract void sound();
}
```

```
class Cat extends Animals
{
    @Override
    void sound()
    {
        System.out.println("Meoww Meoww .....");
    }
}
class Dog extends Animals
{
    @Override
    void sound()
    {
        System.out.println("Woof Woof .....");
    }
}
```

Abstract classes and methods

```
public class AbstractClassCompleteExample
{
    public static void main(String[] args)
    {
        Cat c = new Cat();
        c.sound();
        Dog d= new Dog();
        d.sound();
    }
}
```

ABSTRACT CLASSES & ABSTRACT METHODS



AGENDA

- Interfaces – Introduction
- Implementing Interfaces
- Example

Interfaces - introduction

- ✓ Interface is an abstract type that can contain only the declarations of methods and constants.
- ✓ Interfaces are syntactically similar to classes, but they do not contain instance variables, and their methods are declared without any body.
- ✓ Any number of classes can implement an interface.
- ✓ One class may implement many interfaces.
- ✓ By providing the interface keyword, Java allows to fully utilize the “one interface, multiple methods” aspect of polymorphism.
- ✓ Interfaces are alternative means for multiple inheritance in Java

Interfaces - Defining an Interface

- An interface is defined much like a class.
- This is the general form of an interface:

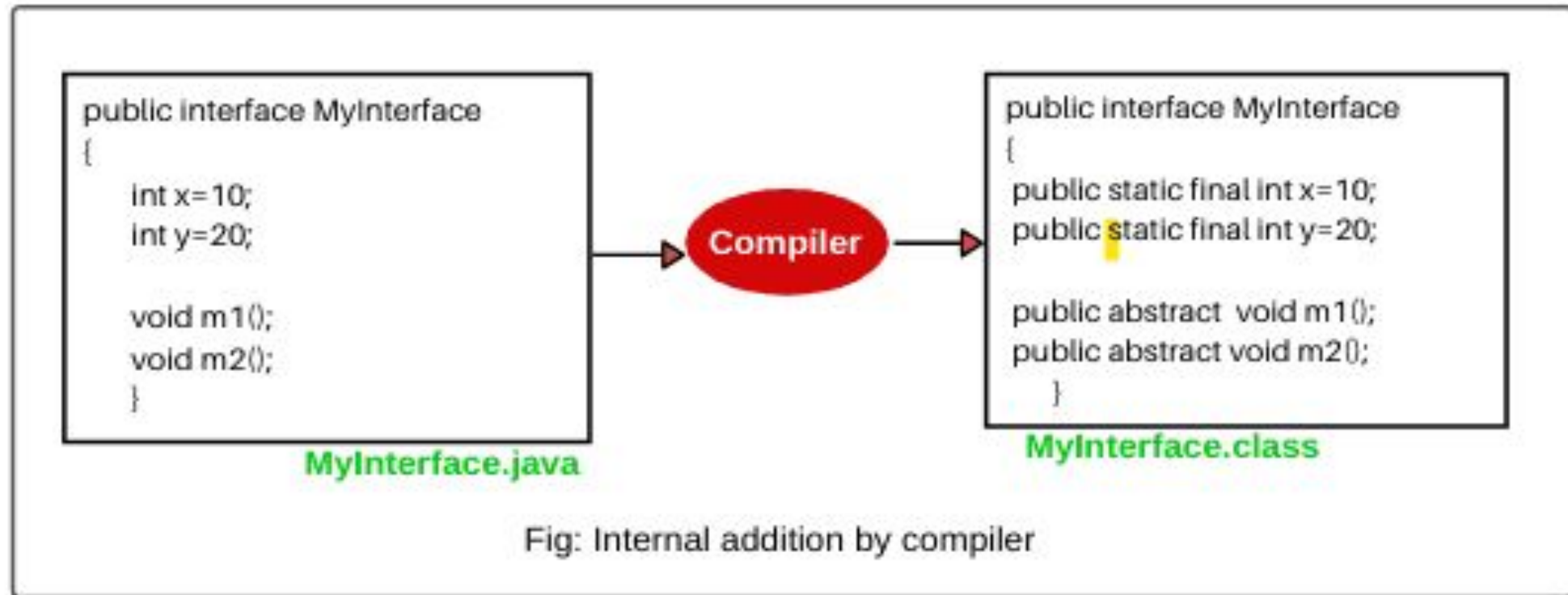
```
<access_specifier> interface name
{
    type final-varname1 = value;
    type final-varname2 = value;
    .....
    return-type method-name1(parameter-list);
    return-type method-name2(parameter-list);
    .....
}
```

Interfaces - introduction

Few key-points about interface:

- When no access specifier is mentioned for an interface, then it is treated as **default** and the interface is only available to other members of the package in which it is declared. When an interface is declared as **public**, the interface can be used by any other code.
- All the methods declared are **abstract** methods and hence are not defined inside interface. But, a class implementing an interface should define all the methods declared inside the interface.
- Variables declared inside of interface are **implicitly final and static**, meaning they cannot be changed by the implementing class.
- All the variables declared inside the interface must be **initialized**.
- All methods and variables are **implicitly public**.

Interfaces - introduction



Interfaces - introduction



Difference between Class & Interface in Java

Class	Interface
Class can instantiate variable & create object	Interface can't instantiate variable & create an object
Class can contain concrete methods	The interface can't contain concrete methods
The access specifiers used with classes are private, protected and public	In interface only one public specifier is used



Interfaces – Implementing interface

- To implement an interface, include the **implements** clause in a class definition, and then create the methods defined by the interface.
- The general form of a class that includes the implements clause looks like this:

```
class classname [extends superclass] [implements interface1[, interface2...] ]  
{  
    // class-body  
}
```

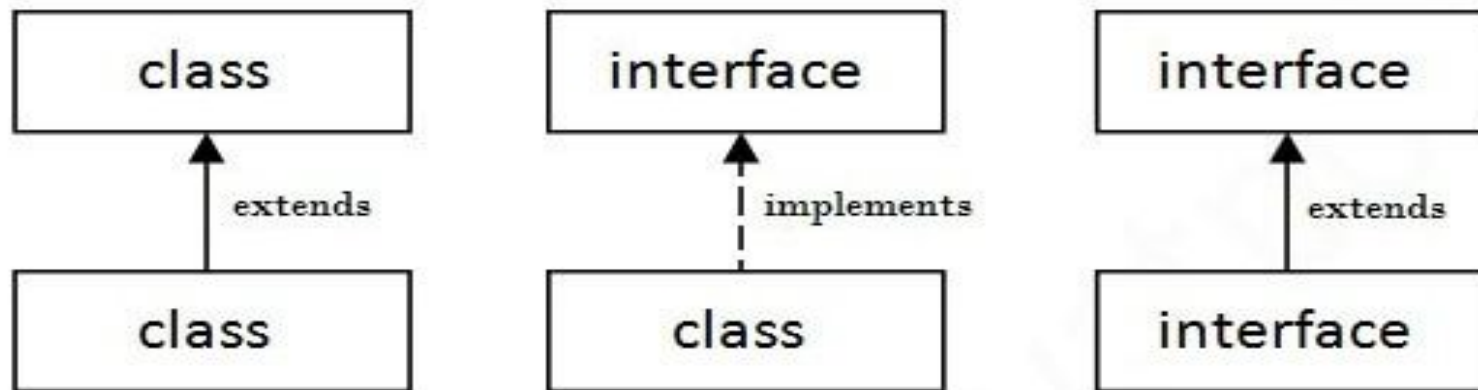
- The methods that implement an interface must be declared public.
- Also, the type signature of the implementing method must match exactly the type signature specified in the interface definition

Interfaces – Implementing interface

Accessing Implementations Through Interface References:

- We can declare variables as object references that use an interface rather than a class type.
- Any instance of any class that implements the declared interface can be referred to by such a variable.
- When you call a method through one of these references, the correct version will be called based on the actual instance of the interface being referred to. This is one of the key features of interfaces

Interfaces – Implementing interface



Interfaces – Implementing interface

Design an Interface called Polygon with a method called area. Implement this Interface to create different classes like Square, Rectangle and print the area of Square and Rectangle.

Recap..

- Interfaces – Introduction
- Implementing Interfaces
- Example

AGENDA

- Nested Interfaces
- Variables in Interfaces
- Extending Interfaces
- Example

Nested Interfaces

- An interface can be declared a member of a class or another interface. Such an interface is called a **member interface** or a **nested interface**.
- A nested interface can be declared as **public**, **private**, or **protected**.
- This differs from a top-level interface, which must either be declared as **public** or use **the default access level**.
- When a nested interface is used outside of its enclosing scope, it must be qualified by the name of the class or interface of which it is a member.
- Thus, outside of the class or interface in which a nested interface is declared, its name must be **fully qualified**.

Variables in Interfaces

- We can use interfaces to import shared constants into multiple classes by simply declaring an interface that contains variables that are initialized to the desired values.
- It is as if that class were importing the constant fields into the class name space as **final variables**.

Extending Interfaces

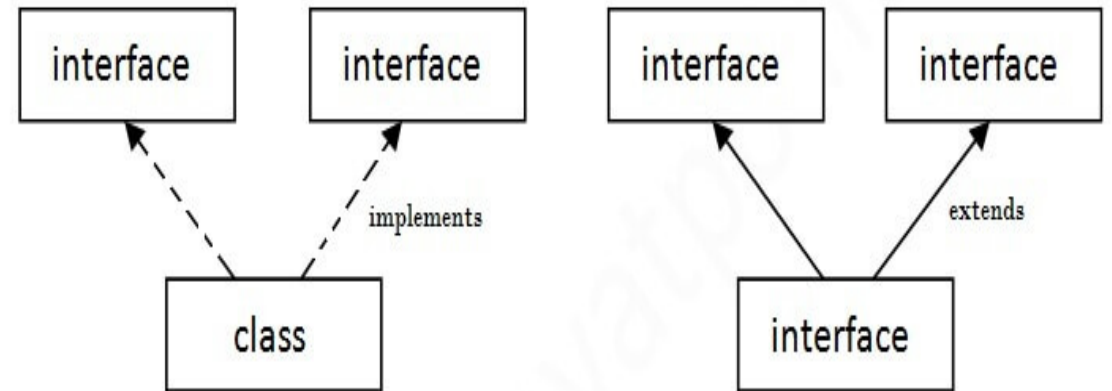
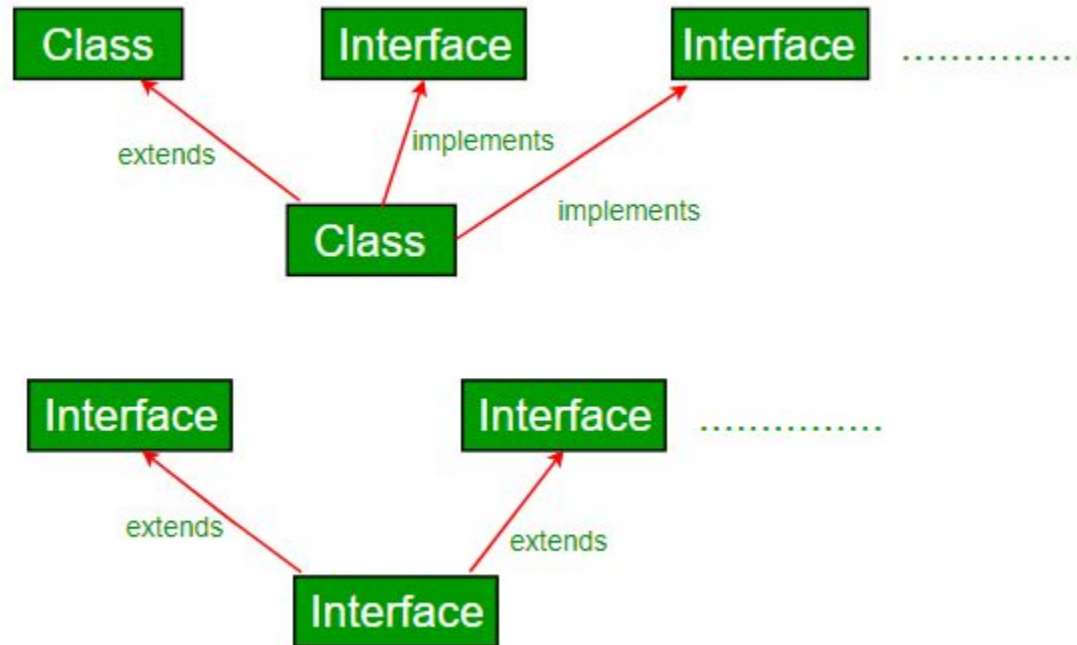
- One interface can inherit another by use of the keyword **extends**.
- **The syntax is the same as for inheriting classes.**
- When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within the interface inheritance chain.

Extending Interfaces



Multiple Inheritance in Java

Extending Interfaces



Multiple Inheritance in Java

Advantages of Interfaces

It is used to achieve abstraction.

1

2

By interface, we can support the functionality of multiple inheritance.

It can be used to achieve loose coupling.

3



PACKAGES

Packages

Packages are Java's way of grouping a number of related classes and/or interfaces together into a single unit. That means, packages act as “containers” for classes.

The benefits of organising classes into packages are:

- The classes contained in the packages of other programs/applications can be reused.
- In packages classes can be unique compared with classes in other packages. That two classes in two different packages can have the same name. If there is a naming clash, then classes can be accessed with their fully qualified name.
- Classes in packages can be hidden if we don't want other packages to access them.
- Packages also provide a way for separating “design” from coding.

Java Foundation Packages

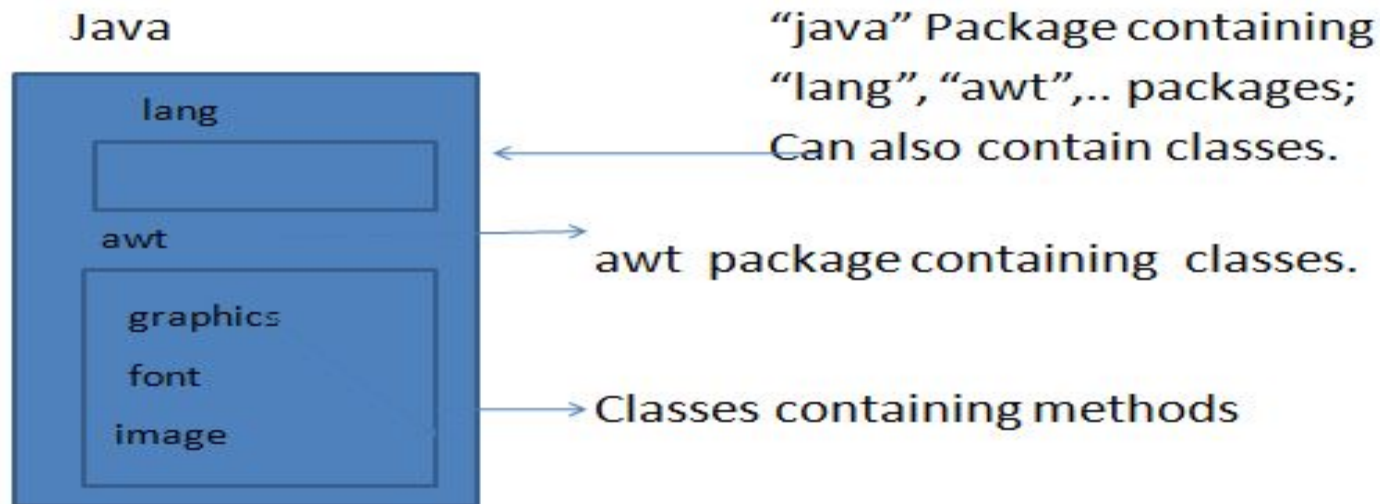
Java provides a large number of classes grouped into different packages based on their functionality

The six foundation Java packages are:

- java.lang :
Contains classes for primitive types, strings, math functions, threads, and exception.
- java.util :
Contains classes such as vectors, hash tables, date etc.
- java.io :
Stream classes for I/O .
- java.awt :
Classes for implementing GUI – windows, buttons, menus etc.
- java.net :
Classes for networking .
- java.applet :
Classes for creating and implementing app.

Using System Packages

The packages are organised in a hierarchical structure .For example, a package named “java” contains the package “awt”, which in turn contains various classes required for implementing GUI (graphical user interface).



Accessing Classes from Packages

There are two ways of accessing the classes stored in packages:

- Using fully qualified class name

```
java.lang.Math.sqrt(x);
```

- Import package and use class name directly

```
import java.lang.Math  
Math.sqrt(x);
```

- Selected or all classes in packages can be imported:

```
import package.class;  
import package.*;
```

Implicit in all programs: `import java.lang.*;`

package statement(s) must appear first

Creating Packages

Java supports a keyword called “package” for creating user-defined packages. The package statement must be the first statement in a Java source file (except comments and white spaces) followed by one or more classes.

```
package myPackage;  
public class ClassA  
{  
    /    / class body  
}  
class ClassB  
{  
}
```

Package name is “myPackage” and classes are considered as part of this package; The code is saved in a file called “ClassA.java” and located in a directory called “myPackage” under current working directory.

Accessing a Package

As indicated earlier, classes in packages can be accessed using a fully qualified name or using a short-cut as long as we import a corresponding package.

The general form of importing package is:

```
import package1[.package2][...].classname
```

Example:

```
import myPackage.ClassA;
```

```
import myPackage.secondPackage
```

All classes/packages from higher-level package can be imported as follows:

```
import myPackage.*;
```


Using a Package

Let us store the code listing below in a file named “ClassA.java” within subdirectory named “myPackage” within the current directory (say “abc”).

```
package myPackage;
public class ClassA
{
    // class body
    public void display()
    {
        System.out.println("Hello, I am ClassA");
    }
}
```

Using a Package

Within the current directory (“abc”) store the following code in a file named “ClassX.java”

```
import myPackage.ClassA;  
public class ClassX  
{  
    public static void main(String args[])  
    {  
        ClassA objA = new ClassA();  
        objA.display();  
    }  
}
```

Compiling and Running

When ClassX.java is compiled, the compiler compiles it and places .class file in current directory directly.

If .class of ClassA in subdirectory “myPackage” is not found, it compiles ClassA also.

Note: It does not include code of ClassA into ClassX

When the program ClassX is run, java loader looks for ClassA.class file in a package called “myPackage” and loads it.

Using a Package

Let us store the code listing below in a file named “ClassC.java” within subdirectory named “secondPackage” within the current directory (say“abc”).

```
public class ClassC {  
    // class body  
    public void display()  
    {  
        System.out.println("Hello, I am ClassC");  
    }  
}
```

Using a Package

Within the current directory (“abc”) store the following code in a file named “ClassX.java”

```
import myPackage.ClassA;
import secondPackage.ClassC;
public class ClassX
{
    public static void main(String args[])
    {
        ClassA objA = new ClassA();
        ClassC objC = new ClassC();
        objA.display();
        objC.display();
    }
}
```

Output

Hello, I am ClassA
Hello, I am ClassC

Access Protection

Access Specifiers in Java

		public	private	protected	default
Same Package	Class	YES	YES	YES	YES
	Sub class	YES	NO	YES	YES
	Non sub class	YES	NO	YES	YES
Different Package	Sub class	YES	NO	YES	NO
	Non sub class	YES	NO	NO	NO